Lecture: N-Queens Problem — Backtracking Masterclass

1. Introduction

The **N-Queens Problem** asks:

Can you place **N queens** on an **N × N chessboard** so that **no two queens attack each other**?

A queen in chess can attack:

- Horizontally (row)
- Vertically (column)
- Diagonally (both directions)

So we must ensure:

- No two queens share the same row
- No two queens share the same column
- No two queens share the same diagonal

2. Example (4-Queens)

For N = 4, one valid solution:

. Q . .

. . . Q

Q . . .

. . Q .

Another solution:

. . Q .

Q . . .

. . . Q

. Q . .

Total solutions for N=4 → **2**

3. Key Idea

Instead of placing queens anywhere randomly:

💡 We place **one queen per row**.

Why?

- Because queens attack horizontally.
- So each recursive level handles one row.

This reduces complexity significantly.

4. Backtracking Strategy

For each row:

1. Try placing a queen in every column.
2. Check if it's **safe**.
3. If safe:
    - Place queen.
    - Recurse to next row.
4. After recursion:
    - Remove queen (backtrack).

5. Safety Check

A position (row, col) is safe if:

- No queen in same column.
- No queen in upper-left diagonal.
- No queen in upper-right diagonal.

Since we place row by row downward, we only check **previous rows**.

6. Recursive Algorithm (Basic Version)

Pseudocode

```
def solve_n_queens(n):
    board = [["."] * n for _ in range(n)]
```

```
result = []
def is_safe(row, col):
    # Check column
    for r in range(row):
        if board[r][col] == "Q":
            return False
# Check upper-left diagonal
    r, c = row - 1, col - 1
    while r >= 0 and c >= 0:
        if board[r][c] == "Q":
            return False
        r -= 1
        c -= 1
# Check upper-right diagonal
    r, c = row - 1, col + 1
    while r >= 0 and c < n:
        if board[r][c] == "Q":
            return False
        r -= 1
        c += 1

    return True

  def backtrack(row):
    if row == n:
        result.append(["".join(r) for r in board])
```

```
        return

    for col in range(n):
        if is_safe(row, col):
            board[row][col] = "Q"
            backtrack(row + 1)
            board[row][col] = "."  # backtrack

  backtrack(0)
  return result
```

7. How Recursion Works (N=4)

Level 0 → Try columns in row 0
Level 1 → Try valid columns in row 1
Level 2 → Continue
Level 3 → Continue
Level 4 → Solution found ✅

Then backtrack and explore other branches.

8. Time Complexity

Aspect	**Complexity**
Worst-case time	O(N!)
Space complexity	$O(N^2)$
Recursion depth	O(N)

Why O(N!)?

- First row → N choices
- Second → at most N-1
- Third → at most N-2

- And so on

9. Optimized Version (Using Sets)

Instead of scanning the board repeatedly, we can track conflicts using sets:

```
def solve_n_queens(n):
    result = []
    cols = set()
    diag1 = set()
    diag2 = set()
    board = [["."] * n for _ in range(n)]

    def backtrack(row):
        if row == n:
            result.append(["".join(r) for r in board])
            return

        for col in range(n):
            if col in cols or (row - col) in diag1 or (row + col) in diag2:
                continue

            cols.add(col)
            diag1.add(row - col)
            diag2.add(row + col)
            board[row][col] = "Q"

            backtrack(row + 1)
```

```
        cols.remove(col)
        diag1.remove(row - col)
        diag2.remove(row + col)
        board[row][col] = "."

    backtrack(0)
    return result
```

✅ Much faster safety checking
✅ Common interview solution

10. Solutions Count for Small N

N	Solutions
1	1
2	0
3	0
4	2
5	10
8	92

11. Variations

Variation	Description
Count only	Return number of solutions
Print one solution	Stop after first
Return board positions	Instead of full board
Bitmask optimization	Used for very large N

12. Common Mistakes

❌ Forgetting to backtrack (remove queen)
❌ Checking full board every time (slow)
❌ Not handling diagonals properly

13. Real-World Applications

Domain	Application
Constraint Satisfaction	AI search problems
Scheduling	Conflict resolution
Chess engines	Move validation
Puzzle solvers	State space exploration

14. Key Takeaways

- Place **one queen per row**
- Use **backtracking**
- Use **sets for optimization**
- Problem complexity grows factorially

15. Final Understanding

The **N-Queens problem** teaches:

✓ How to reduce problem space
✓ How to apply constraints early
✓ How to prune invalid branches
✓ How recursion explores possibilities

It is one of the strongest problems to master backtracking 🔥